

Effa Ahsan
27100463

Task 8: Documentation & Architecture Diagram (5 Points)

Include a short write-up covering:

Service selection: explain the choices. For each of App Service, Durable Functions, Azure Kubernetes Service (AKS), and Container Instances, write 3-4 sentences explaining why it is the right tool for its specific role in TaskFlow. Be specific about cost, scale behaviour, and operational model.

App Service hosts the taskflow web front-end because it is designed for always-on web applications with built-in GitHub CI/CD. It runs on a B1 Basic plan which provides dedicated compute at low cost without cold-start delays.

Durable Functions coordinate the pipeline because the workflow has multiple async steps that require state persistence between them. A plain HTTP function would time out during long-running report generation and lose all progress if it crashed. KS hosts the validator because it is a long-lived HTTP microservice that must respond instantly to every order. Keeping it always running on a dedicated node ensures zero cold-start latency when the Function App calls it synchronously. ACI runs the report generator because it is a short-term microservice which writes a PDF, and exits. It bills only while running, making it far cheaper than keeping an always-on AKS pod idle between orders.

2. ACI vs. AKS: hands-on comparison. Using your AKS cluster metrics and the az container list screenshot from Task 7.2, explain: What happens to AKS when it is idle for 10 minutes? What does idle even mean for ACI in your pipeline? If a malicious user spammed the Submit button 1000 times in a minute, which service would incur the most cost, and why?

Nothing happens to AKS as it keeps running and keeps billing. The pod stays alive, the node continues consuming compute and the validator is ready to respond instantly. ACI has no such thing as 'idle' as it simply does not exist between orders. The Function App creates a new ACI instance per order, it generates the PDF and uploads it to blob storage and is then deleted.

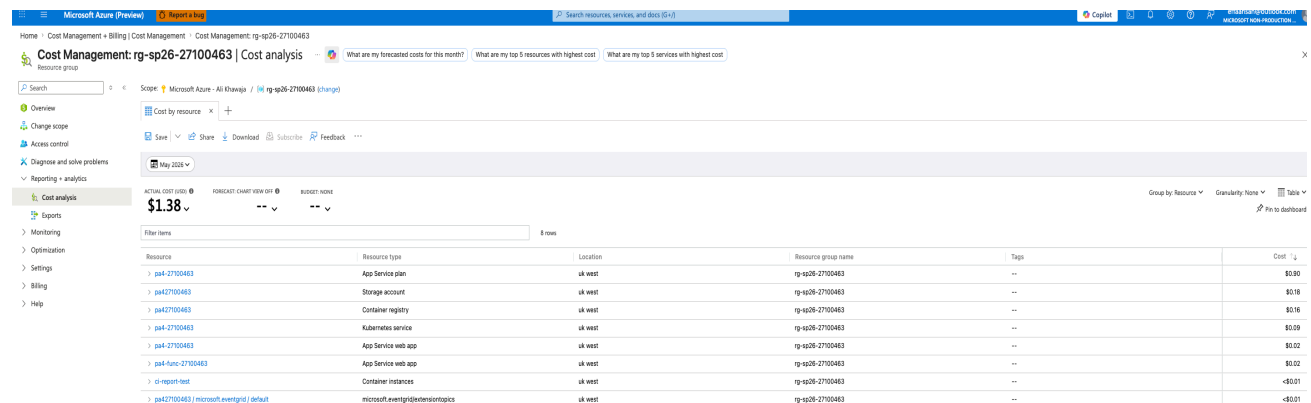
ACI would incur the most cost because each order submission triggers the creation of a new ACI container, meaning 1000 containers would have to be created each billing for their runtime. AKS on the other hand would just receive 1000 HTTP requests to the validator on the same single node.

3. Durable Functions: why not just HTTP? Your orchestrator chains validate and report, and the report step takes up to a minute. Explain in 3-4 sentences what would be harder if you implemented the same ow as two plain HTTP-triggered functions that called each other. Name at least two concrete problems (hint: function timeouts, state persistence, retry-on-failure).

The HTTP-triggered functions have a maximum timeout-limit of 10 minutes so incase for example if any process such as the report generation in this task takes longer than 10 minutes the function will timeout and since there is no built-in state persistence no state is preserved in between the orders and the entire order gets lost. Incase of any failure of the function there is no retry-on failure option and the entire order will have to get processed from the very start.

4. Cost review. Screenshot Cost Management → Cost Analysis scoped to your resource group. Estimate what this PA consumed in the instructor's subscription. Identify the single most expensive resource.

Most expensive resource: App Service Plan (\$0.90) The App Service Plan is the biggest cost driver because it runs 24/7 and hosts both the Web App and Function App on the same B1 Basic instance. Interestingly, the AKS Kubernetes service only cost \$0.09 because it was running for a shorter time period



The screenshot shows the Microsoft Azure Cost Management interface. The main heading is "Cost Management: rg-sp26-27100463 | Cost analysis". Below this, there's a table of resources with columns: Resource, Resource type, Location, Resource group name, Tags, and Cost. The table lists 8 resources, with the App Service plan being the most expensive at \$0.90.

Resource	Resource type	Location	Resource group name	Tags	Cost
pa4-27100463	App Service plan	uk west	rg-sp26-27100463	--	\$0.90
pa427100463	Storage account	uk west	rg-sp26-27100463	--	\$0.18
pa427100463	Container registry	uk west	rg-sp26-27100463	--	\$0.16
pa4-27100463	Kubernetes service	uk west	rg-sp26-27100463	--	\$0.09
pa4-27100463	App Service web app	uk west	rg-sp26-27100463	--	\$0.02
pa4-func-27100463	App Service web app	uk west	rg-sp26-27100463	--	\$0.02
ci-report-test	Container instances	uk west	rg-sp26-27100463	--	<\$0.01
pa427100463 microsoft.eventgrid default	microsoft.eventgridextensions	uk west	rg-sp26-27100463	--	<\$0.01

Screenshot

Challenges faced. Describe at least two issues you hit and how you debugged them. The grader values honesty over everything worked.

Web App running PHP instead of Node.js After deploying the Function App container, the Web App started serving a PHP runtime instead of the Node.js app from GitHub. This happened because the `az functionapp create` command accidentally overwrote the existing Web App's configuration with Docker container settings, changing the runtime from Node to PHP. The error was discovered by checking the live logs at `pa427100463.scm.azurewebsites.net/api/logstream` which showed PHP version: 8.0.30 and `npm: not found`. The fix required deleting the Docker container configuration, resetting the runtime to `NODE:22-lts`, zipping the `webapp/` subfolder correctly in the GitHub Actions workflow, and redeploying.

Azure CLI authentication issues Throughout the assignment, the Azure CLI kept switching to the wrong subscription (MCAPS-Support instead of Microsoft Azure - Ali Khawaja). This caused `AuthorizationFailed` errors on almost every command. The workaround was to run `az account set --subscription 67e93b84-fe08-452c-80ea-175d0a3eca56` at the start of every Cloud Shell session. Additionally, the CLI consistently showed null values for app settings even when they were correctly saved in the portal, requiring all environment variables to be set manually through the Azure Portal instead.